异或症测试 4

xor | 进位掩码、逐位贪心与 SOS DP | C++17

1. 实验值就是进位掩码

对选定的两个数 a、b，记 a_i、b_i 为二进制第 i 位，c_i 为进入第 i 位的进位，c_0=0。普通加法第 i 位满足下式：

```
(a+b)_i = a_i xor b_i xor c_i
```

因此 (a xor b) xor (a+b) 的第 i 位恰为 c_i。目标就是最大化整个进位掩码。输入数小于 2^m，可能有意义的输出位为 1 到 m；第 0 位恒为零。

2. 固定相邻进位后的限制

加法进位规则是：a_i、b_i、c_i 中至少两个为 1 时，c_(i+1)=1。固定 c_i 和 c_(i+1)，可得到四种限制：

```
c_i c_(i+1) constraint on (a_i,b_i)
0 0 not (1,1)
0 1 must be (1,1)
1 0 must be (0,0)
1 1 not (0,0)
```

3. 从高位到低位贪心

依次处理 k=m,m-1,...,1。保持已确定的高位进位可由某一对输入数实现，尝试令 c_k=1。若存在满足该高位前缀的数对，就保留这一位；否则令其为 0。

因为更高位的价值大于全部更低位之和，且每次都使用精确的存在性判定，该贪心最终得到最大实验值。下面构造这个判定。

把高位限制变成不相交

4. 统一四种逐位条件

候选掩码 C 的第 k 到 m 位已经固定，低于 k 的位暂置零。对每个 i>=k，定义 u_i=a_i xor c_(i+1)，v_i=b_i xor c_(i+1)。

当 c_i=c_(i+1) 时，上一页的两种稳定进位情况统一成“u_i、v_i 不能同时为 1”。当两位进位不同，则统一成“u_i=v_i=0”。

```
u = (a >> k) xor (C >> (k+1))
v = (b >> k) xor (C >> (k+1))
B = ((C xor (C >> 1)) >> k) & ((1 << (m-k))-1)
```

B 标记进位发生变化的高位。在这些位置，每个候选数自己的变换后高位必须为 0。所有高位的完整限制就等价于：

```
(u & B) == 0
(v & B) == 0
(u & v) == 0
```

5. 第 k 位进位由低 k 位之和决定

记 A=a mod 2^k，D=b mod 2^k。进入第 k 位的进位为 1，当且仅当低 k 位加法溢出到这一位，即 A+D>=2^k。

对每种合格的变换后高位掩码 u，只需保留该组中最大的低 k 位数值 g[u]。因为判定低位之和是否达到阈值时，较大的低位数值不会更差。不存在该组时记为 -1。

```
g[u] = maximum (a & ((1 << k)-1))
among eligible input values with mask u
```

于是判定变为：能否找到两个不相交的合格高位掩码，其对应低位最大值之和至少为 2^k。

子集最大值与完整判定

6. SOS DP

设 h=m-k，U=2^h-1。将 g 做子集最大值变换，得到 F[x]：所有 x 的子掩码对应的 g 值中的最大值。

```
F[x] = max(g[s] : s is a submask of x)
for each bit b:
for x having bit b:
F[x] = max(F[x], F[x without bit b])
```

枚举 x，检查 F[x] 与 F[U xor x] 是否均存在，且两者之和达到 2^k。这两个值分别对应 x 与其补集的子掩码，所以选出的两个高位掩码一定不相交。

反过来，任何不相交的高位掩码 u、v，都可以被某个互补划分覆盖，例如取 x=u，则 u 是 x 的子掩码，v 是 U xor x 的子掩码。F 取最大值不会使低位之和减小。因此该判定既不漏解，也不引入非法解。

7. 为什么不需要限制两个下标不同

原题允许 i=j，因此同一个输入数可以被两组最大值同时选到。特别地，变换后掩码为零时，它与自身不相交，重复使用该数完全合法。重复数值也只需按数值存在性处理。

8. 贪心不变量

初始没有固定进位时，总有数对可选。尝试 c_k=1 成功，就选取其合法前缀。尝试失败，则当前前缀下所有合法数对都只能令 c_k=0，所以回退到 0 仍保持可行。处理至 k=1 后，c_0 自动为 0，全部输出位已经确定。

复杂度与实现检查

9. 时间与空间

处理第 k 位时，扫描 n 个输入数，使用 2^(m-k) 个 DP 状态，SOS DP 用 O((m-k)×2^(m-k)) 时间。把所有位相加：

```
time = O(n*m + m*2^m)
space = O(n + 2^m)
```

利用 m<=20、所有测试的 n 之和不超过 10^6、2^m 之和不超过 2^22，总体操作量受 O(20×10^6+20×2^22) 控制。每一位都重新初始化其实际使用的 DP 区间，不对所有小测试固定清空大数组。

10. 边界与数值

m=0 时输入只能为零，没有需要贪心的位，直接输出 0。n=1 时仍可选择自身，例如 a=7 时答案为 14。最大答案小于 2^(m+1)，普通 int 足够。

候选不存在时用 -1，并且仅在两个 DP 值均非负时比较和。不要把普通加法截断为 m 位，最高进位 c_m 必须保留。代码默认使用 xor.in / xor.out，加 -DLOCAL 可使用标准输入输出。

11. 例子与验证

a=3、b=5 时，a xor b=6，a+b=8，两者异或为 14，二进制进位掩码是 1110。

题面三组样例输入全部通过，两种 IO 模式均核对。额外枚举数值集合 {0,...,15} 的所有 65535 个非空子集，并做 20000 组随机小数据，每组用枚举数对直接求实验值作独立检查，共 85535 组通过。随机种子 20261011。

另完成 n=10^6、m=20 的大规模运行检查。验证结果用于核对实现，正确性依据是逐位进位等价、存在性判定及贪心证明；未经官方评测。